UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

ESCUELA DE INGENIERÍA DE SISTEMAS



Guía Práctica de Laboratorio

Implementación de Room Database en Kotlin

Que se presenta para el curso:

SOLUCIONES MÓVILES I

AUTOR(ES):

APELLIDOS Y NOMBRES

Docente:

Mag. Alberto Johnatan Flor Rodríguez

TACNA – PERÚ

2026

Índice general

Introducción **1**

1. Guía de Laboratorio No. 06: Implementación de Room Database en Kotlin **2**

1.1. Información sobre el evento práctico . . . 2

1.1.1. Título del evento práctico . . . 2

1.1.2. Objetivos . . . 2

1.1.3. Tiempo de duración . . . 2

1.1.4. Resultados de aprendizaje . . . 2

1.1.5. Recursos . . . 2

1.2. Procedimiento o metodología . . . 3

1.2.1. Inicio de la aplicación . . . 3

1.2.2. Nombre del proyecto . . . 3

1.2.3. Clases y disposición de la aplicación . . . 4

1.2.4. Configuración de Gradle, Kotlin y KSP . . . 6

1.2.5. Entidad UserEntity . . . 8

1.2.6. DAO . . . 8

1.2.7. Base de datos e instancia de aplicación . . . 9

1.2.8. AndroidManifest.xml . . . 9

1.2.9. ViewModel y corrutinas . . . 10

1.2.10. Actualización, eliminación y consulta . . . 10

2. Ampliación implementada **12**

2.1. Enunciado . . . 12

2.2. Entidad y migración . . . 12

2.3. Búsqueda por apellido y lista reactiva . . . 13

2.4. Interfaz de usuario . . . 13

2.5. Worker demostrativo . . . 14

2.6. Capturas de la implementación nueva 14

3. Conclusiones **17**

4. Cuestionario **18**

1. Función de @Dao 18

2. Clave primaria autogenerada con valor 0 18

3. DAO, ViewModel y Dispatchers.IO 18

4. Propósito de Application 18

5. Función de KSP con Room 19

5. Comprobación de compilación **20**

6. Referencias bibliográficas **21**

Introducción

En el desarrollo de aplicaciones Android modernas, la persistencia de datos es un pilar fundamental para ofrecer una experiencia de usuario fluida, permitiendo que la información se mantenga disponible incluso sin conexión a internet o tras cerrar la aplicación. Room Database se presenta como una capa de abstracción sobre SQLite que facilita el acceso a la base de datos, aprovechando la potencia de SQLite mientras proporciona verificación de consultas en tiempo de compilación.

Este laboratorio práctico se centra en la arquitectura de persistencia recomendada por Google. Se utilizan los tres componentes clave de Room:

1. **Entity:** representa las tablas de la base de datos.
2. **DAO (Data Access Object):** define los métodos para acceder a los datos.
3. **Database:** sirve como punto de acceso principal a la conexión.

Además, se integra el uso de corrutinas de Kotlin y el componente ViewModel para que las operaciones de entrada/salida se realicen de forma asíncrona, manteniendo la interfaz de usuario reactiva y libre de bloqueos.

Capítulo 1

Guía de Laboratorio No. 06: Implementación de Room Database en Kotlin

1.1. Información sobre el evento práctico

1.1.1. Título del evento práctico

Implementación de Room Database en Kotlin.

1.1.2. Objetivos

- Implementar Room como medio de almacenamiento interno de información.
- Integrar Workers con corrutinas mediante `Dispatchers.IO` para optimizar el uso de hilos.

1.1.3. Tiempo de duración

2 horas.

1.1.4. Resultados de aprendizaje

- Gestionar tareas de larga duración fuera del hilo principal y garantizar la persistencia de procesos aunque la aplicación se cierre.
- Aplicar la separación de responsabilidades delegando la lógica de trabajo pesado a Workers especializados.

1.1.5. Recursos

- Computadora con acceso a internet.

- Android Studio.
- Dispositivo Android físico o un Android Virtual Device (AVD).

1.2. Procedimiento o metodología

1.2.1. Inicio de la aplicación

Se crea un proyecto nuevo en Android Studio con la plantilla *Empty Activity*.

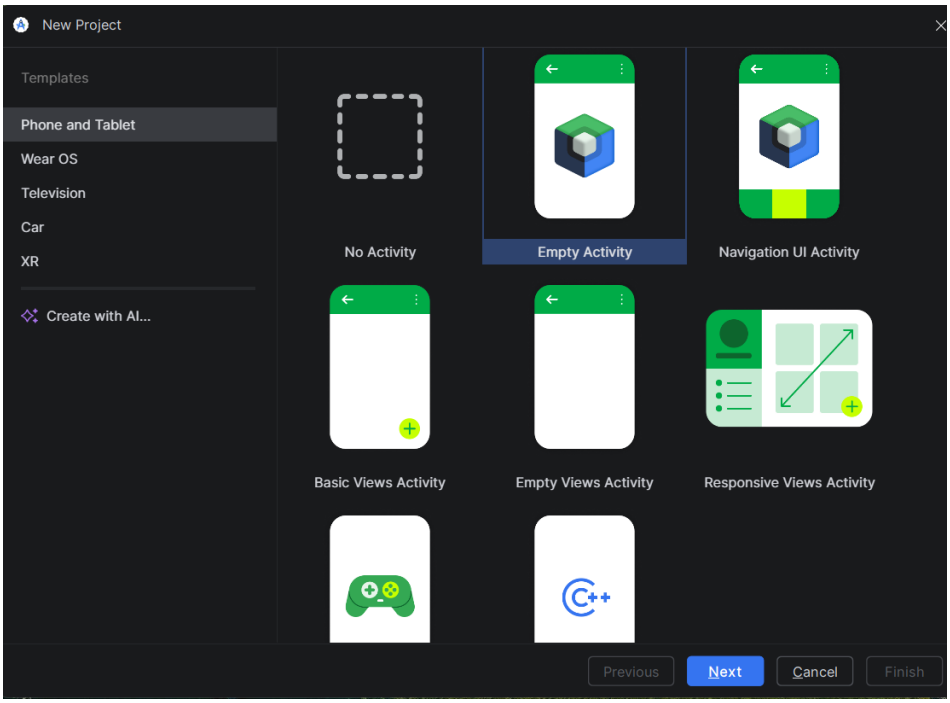


Figura 1.1: Selección de la plantilla Empty Activity en Android Studio.

1.2.2. Nombre del proyecto

Se asigna el nombre `roomDatabase`, el paquete `com.example.roomdatabase` y el lenguaje Kotlin.

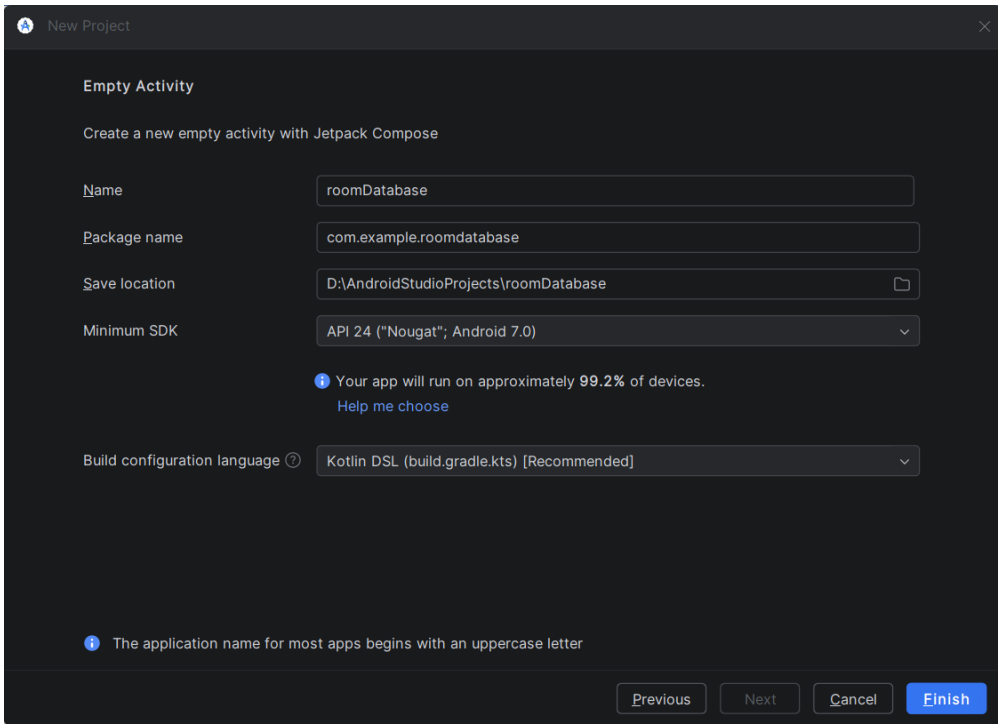


Figura 1.2: Configuración inicial del proyecto roomDatabase.

1.2.3. Clases y disposición de la aplicación

Se propone la siguiente distribución de clases para cubrir la lógica de mantenimiento de tablas mediante Room:

```
@startuml
package "UI Layer" {
  class MainActivity {
    - viewModel: MainViewModel
    + onCreate()
  }
  class MainViewModel {
    + insertUser()
    + insertUserList()
    + updateUser()
    + deleteUser()
    + getAllUsers()
  }
}
package "Application" {
  class RoomApplication {
    {static} + db: AppDatabase
    + onCreate()
  }
}
package "Data Layer (Room)" {
  abstract class AppDatabase {
    + {abstract} userDao(): UserDao
```

```
  }
  interface UserDao <<interface>> {
    + insertUser(user: UserEntity): Long
    + insertUserList(people: List<UserEntity>): List<Long>
    + updateUser(user: UserEntity): Int
    + deleteUser(user: UserEntity): Int
    + getAll(): List<UserEntity>
    + getAllById(userIds: IntArray): List<UserEntity>
  }
  class UserEntity <<data>> {
    + userId: Int
    + nombres: String?
    + apellidos: String?
  }
}
MainActivity --> MainViewModel : usa
MainViewModel --> RoomApplication : accede db
RoomApplication o-- AppDatabase : mantiene instanciado
AppDatabase ..> UserDao : provee
UserDao ..> UserEntity : opera con
@enduml
```

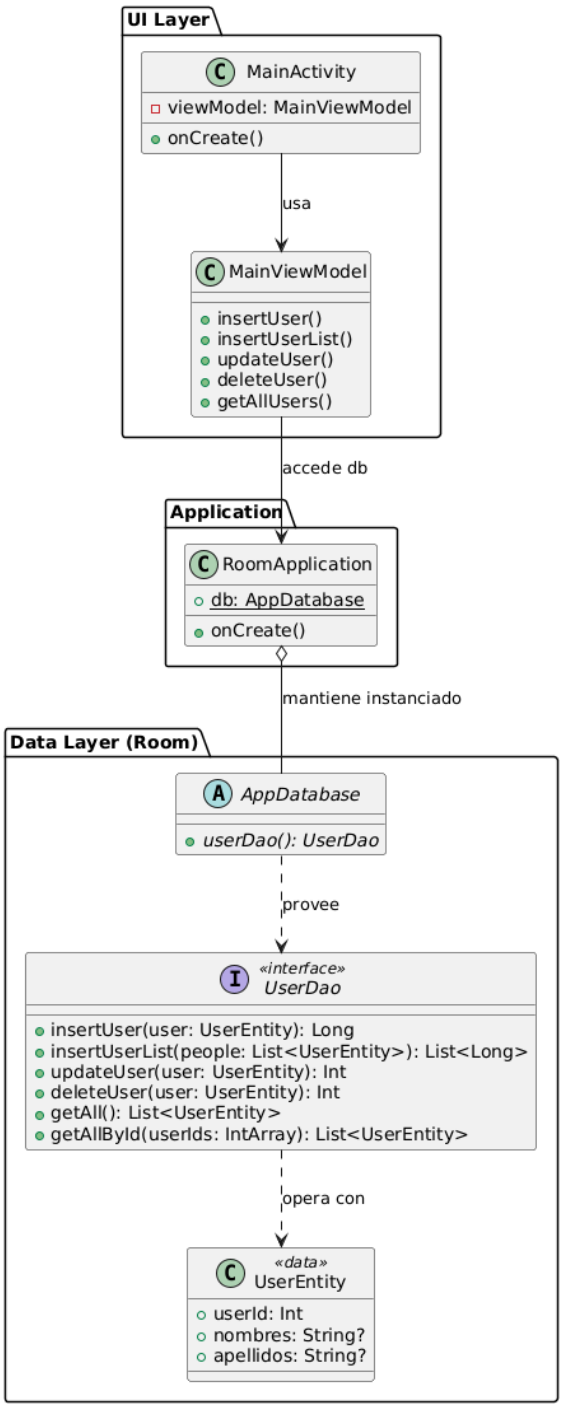


Figura 1.3: Diagrama de clases y relaciones de la solución base.

1.2.4. Configuración de Gradle, Kotlin y KSP

Se modifican los archivos de Gradle para configurar el compilador, KSP y las dependencias de Room. Las versiones indicadas por la guía son Kotlin 2.3.21, Compose BOM 2026.04.01, KSP 2.3.4 y Room 2.8.4.

```
[versions]
kotlin = "2.3.21"
```

```
composeBom = "2026.04.01"
kotlinskp = "2.3.4"

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref =
    "kotlin" }
kotlinksp = { id = "com.google.devtools.ksp", version.ref = "kotlinskp" }
```

En el Gradle del proyecto se declara KSP sin aplicarlo al módulo raíz:

```
plugins {
  alias(libs.plugins.kotlinksp) apply false
}
```

En el Gradle de la aplicación se aplica el plugin y se agregan las dependencias:

```
plugins {
  alias(libs.plugins.kotlinksp)
}

dependencies {
  val room_version = "2.8.4"
  implementation("androidx.room:room-runtime:$room_version")
  ksp("androidx.room:room-compiler:$room_version")
  annotationProcessor("androidx.room:room-compiler:$room_version")
  implementation("androidx.room:room-ktx:$room_version")
}
```

Se crean los paquetes principales: `application`; `data`, con los subpaquetes `dao`, `database` y `entity`; además de `ui` y `viewmodel`.

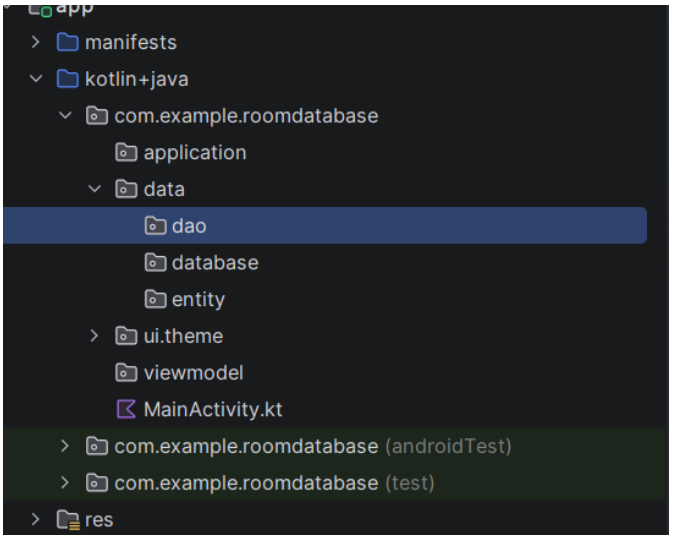


Figura 1.4: Estructura de paquetes mostrada en Android Studio en la guía original.

1.2.5. Entidad UserEntity

La entidad representa la tabla `users`. El identificador se genera automáticamente y los campos de nombre se guardan en columnas con nombres explícitos.

```
package com.example.roomdatabase.data.entity

import androidx.room.ColumnInfo
import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "users")
data class UserEntity(
  @PrimaryKey(autoGenerate = true) val userId: Int = 0,
  @ColumnInfo(name = "first_name") val nombres: String?,
  @ColumnInfo(name = "last_name") val apellidos: String?
)
```

1.2.6. DAO

La interfaz `UserDao` define las operaciones de inserción, actualización, eliminación y consulta:

```
package com.example.roomdatabase.data.dao

import androidx.room.Dao
import androidx.room.Delete
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Update
import com.example.roomdatabase.data.entity.UserEntity

@Dao
interface UserDao {
  @Insert(onConflict = OnConflictStrategy.ABORT)
  suspend fun insertUser(user: UserEntity): Long

  @Insert(onConflict = OnConflictStrategy.ABORT)
  suspend fun insertUserList(people: List<UserEntity>): List<Long>

  @Update
  suspend fun updateUser(user: UserEntity): Int

  @Delete
  suspend fun deleteUser(user: UserEntity): Int

  @Query("SELECT * FROM users")
  suspend fun getAll(): List<UserEntity>
```

```
  @Query("SELECT * FROM users WHERE userId IN (:userIds)")
  suspend fun getAllById(userIds: IntArray): List<UserEntity>?
}
```

1.2.7. Base de datos e instancia de aplicación

`AppDatabase` registra la entidad y ofrece acceso al DAO:

```
@Database(entities = [UserEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
  abstract fun userDao(): UserDao
}
```

La clase que hereda de `Application` crea la instancia de la base de datos:

```
class RoomApplication : Application() {
  companion object {
    lateinit var db: AppDatabase
  }

  override fun onCreate() {
    super.onCreate()
    db = Room.databaseBuilder(
      applicationContext,
      AppDatabase::class.java,
      "database-users"
    ).build()
  }
}
```

1.2.8. AndroidManifest.xml

Se indica la clase `RoomApplication` en el manifiesto:

```
<application
  android:name=".application.RoomApplication"
  android:allowBackup="true"
  android:label="@string/app_name"
  android:supportsRtl="true"
  android:theme="@style/Theme.RoomDatabase">
  <activity
    android:name=".MainActivity"
    android:exported="true">
    <intent-filter>
      <action android:name="android.intent.action.MAIN" />
      <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
  </activity>
```

```
</application>
```

1.2.9. ViewModel y corrutinas

El ViewModel ejecuta las operaciones de base de datos dentro de `viewModelScope` y `Dispatchers.IO`. El ejemplo inserta un usuario y un lote de cinco usuarios:

```
class MainViewModel : ViewModel() {
  fun insertUser() {
    val user = UserEntity(nombres = "Juan", apellidos = "Perez")
    viewModelScope.launch(Dispatchers.IO) {
      RoomApplication.db.userDao().insertUser(user)
    }
  }

  fun insertUserList() {
    val users = listOf(
      UserEntity(nombres = "Juan", apellidos = "Perez"),
      UserEntity(nombres = "Maria", apellidos = "Lopez"),
      UserEntity(nombres = "Pedro", apellidos = "Gonzalez"),
      UserEntity(nombres = "Ana", apellidos = "Martinez"),
      UserEntity(nombres = "Luis", apellidos = "Garcia")
    )
    viewModelScope.launch(Dispatchers.IO) {
      RoomApplication.db.userDao().insertUserList(users)
    }
  }
}
```

La actividad obtiene el ViewModel con `by viewModels()` y, en la solución base de la guía, invoca sus operaciones desde `onCreate()`.

1.2.10. Actualización, eliminación y consulta

Los métodos restantes de mantenimiento actualizan, eliminan y consultan usuarios:

```
fun updateUser() {
  val user = UserEntity(userId = 1, nombres = "Jorge", apellidos = "Perez")
  viewModelScope.launch(Dispatchers.IO) {
    RoomApplication.db.userDao().updateUser(user)
    println("DEBUG: Usuario actualizado correctamente")
  }
}

fun deleteUser() {
  val user = UserEntity(userId = 4, nombres = "", apellidos = "")
  viewModelScope.launch(Dispatchers.IO) {
    RoomApplication.db.userDao().deleteUser(user)
  }
```

```
}

fun getAllUsers() {
  viewModelScope.launch(Dispatchers.IO) {
    val users = RoomApplication.db.userDao().getAll()
    users.forEach { user ->
      println("DEBUG: $user")
    }
  }
}
```

Capítulo 2

Ampliación implementada

2.1. Enunciado

Una vez implementada la estructura básica de `UserEntity` y su mantenimiento, se escala la aplicación para que sea funcional para un usuario final, añadiendo validaciones y una visualización de datos más robusta.

1. **Modificación de la entidad.** Agregar el campo nullable `email`; actualizar `AppDatabase` a la versión 2 y configurar la política de migración destructiva básica solicitada.
2. **Búsqueda:** añadir una consulta por apellido (`last_name`) usando `LIKE` e implementarla en el ViewModel.
3. **Interfaz:** crear en Jetpack Compose los campos Nombre y Apellido, el botón Guardar Usuario y una lista LazyColumn o RecyclerView.
4. **Lógica de negocio:** actualizar la lista automáticamente al insertar o eliminar registros.

2.2. Entidad y migración

La versión final de la entidad incorpora el correo opcional. La versión de la base de datos es 2; al actualizar desde un esquema anterior sin una migración explícita, la política solicitada reconstruye las tablas.

```
@Entity(tableName = "users")
data class UserEntity(
  @PrimaryKey(autoGenerate = true) val userId: Int = 0,
  @ColumnInfo(name = "first_name") val nombres: String?,
  @ColumnInfo(name = "last_name") val apellidos: String?,
  @ColumnInfo(name = "email") val email: String? = null
)

@Database(entities = [UserEntity::class], version = 2, exportSchema = true)
```

```
abstract class AppDatabase : RoomDatabase() {
  abstract fun userDao(): UserDao
}

db = Room.databaseBuilder(
  applicationContext,
  AppDatabase::class.java,
  "database-users"
).fallbackToDestructiveMigration().build()
```

2.3. Búsqueda por apellido y lista reactiva

El DAO expone una consulta con `LIKE` como `Flow`. El ViewModel selecciona la consulta general o la búsqueda según el texto ingresado; Room vuelve a emitir la lista cuando cambian las filas de `users`.

```
@Query(
  "SELECT * FROM users " +
  "WHERE last_name LIKE '%' || :query || '%' " +
  "ORDER BY userId DESC"
)
fun searchByLastName(query: String): Flow<List<UserEntity>>

val users: StateFlow<List<UserEntity>> = _lastNameQuery
  .flatMapLatest { query ->
    if (query.isBlank()) userDao.observeAll()
    else userDao.searchByLastName(query.trim())
  }
  .flowOn(Dispatchers.IO)
  .stateIn(
    viewModelScope,
    SharingStarted.WhileSubscribed(5_000),
    emptyList()
  )
```

2.4. Interfaz de usuario

La pantalla final incluye los dos campos solicitados, validación de campos vacíos, alta y edición de usuarios, eliminación con confirmación, búsqueda por apellido y una lista reactiva. El campo `email` permanece opcional en la entidad y no se solicita en el formulario, que conserva los dos campos requeridos por el enunciado.

```
OutlinedTextField(
  value = firstName,
  onValueChange = { firstName = it },
  label = { Text("Nombre") },
```

```
  singleLine = true
)
OutlinedTextField(
  value = lastName,
  onValueChange = { lastName = it },
  label = { Text("Apellido") },
  singleLine = true
)
Button(onClick = {
  viewModel.saveUser(firstName, lastName)
}) {
  Text("Guardar Usuario")
}
LazyColumn {
  items(items = users, key = { it.userId }) { user ->
    UserCard(user = user, onEdit = { /* cargar usuario */ },
      onDelete = { /* eliminar usuario */ })
  }
}
```

2.5. Worker demostrativo

Los objetivos y resultados de aprendizaje de la guía mencionan Workers, aunque el procedimiento original no especifica una tarea concreta. Para cubrir ese objetivo, se añadió el botón **Cargar lote de ejemplo con WorkManager**. Este encola un `CoroutineWorker` idempotente que inserta los cinco usuarios de ejemplo en `Dispatchers.IO`; la lista observada por Room se actualiza al completar la operación.

2.6. Capturas de la implementación nueva

Las capturas se tomaron desde un AVD Pixel 7 Pro después de compilar e instalar la app de depuración. La primera muestra la pantalla y los registros locales; la segunda demuestra el filtro por apellido `Lopez`.

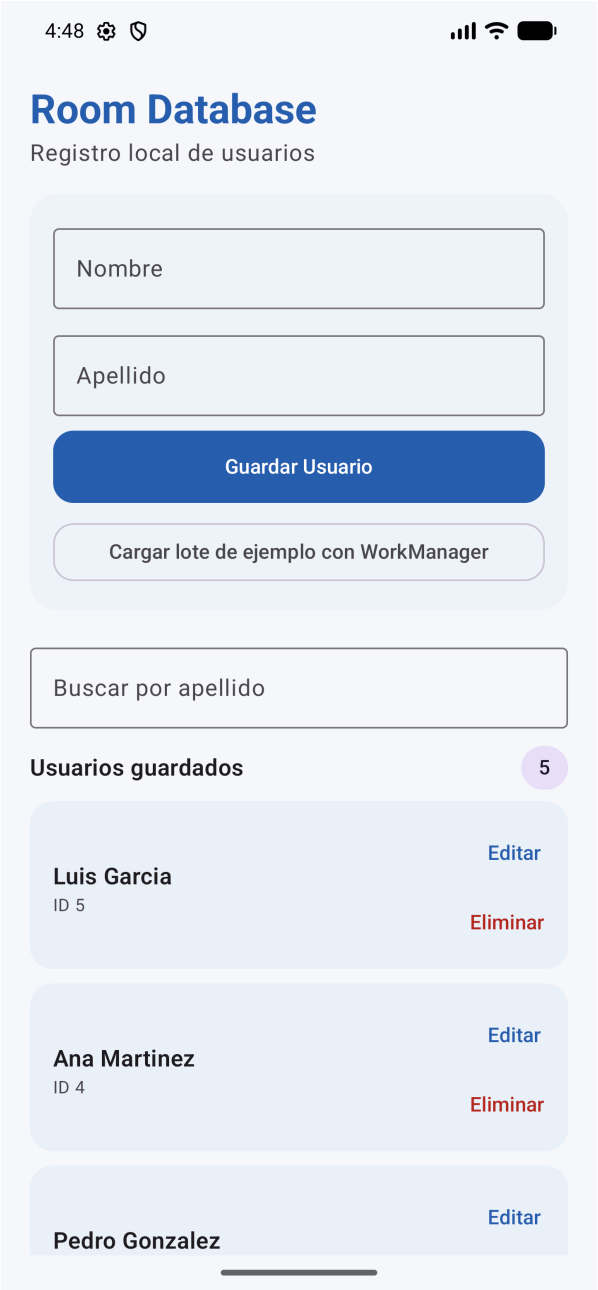


Figura 2.1: Pantalla Compose con usuarios guardados en Room y acciones de edición y eliminación.

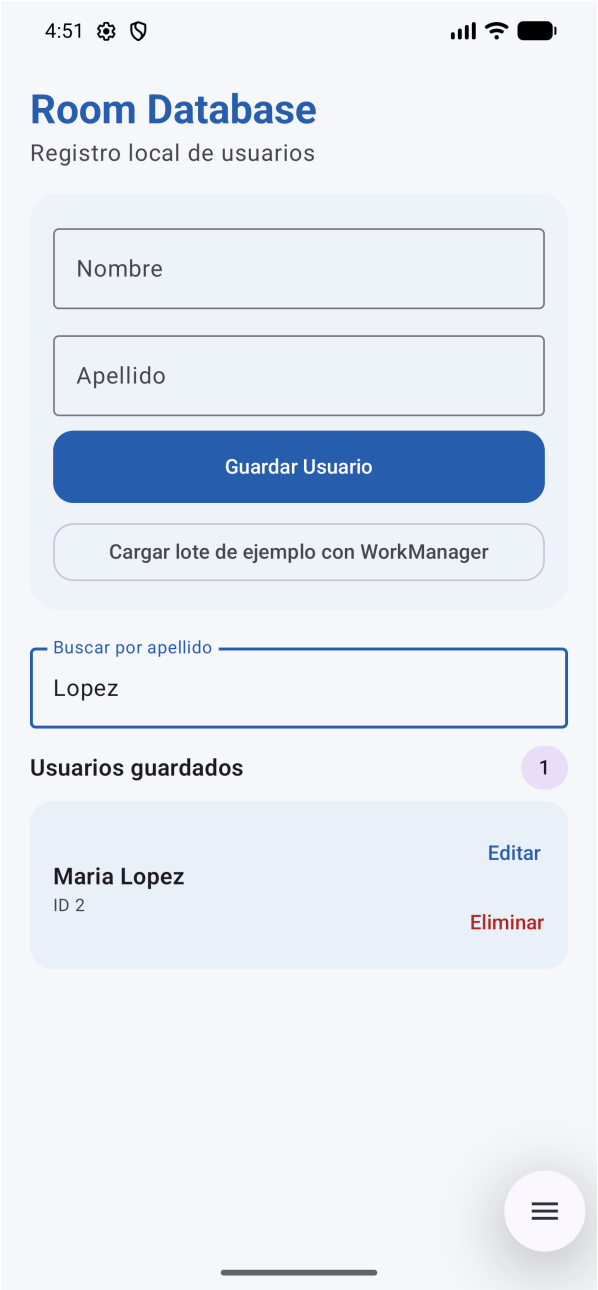


Figura 2.2: Búsqueda reactiva por apellido: al buscar Lopez se muestra el usuario coincidente.

Capítulo 3

Conclusiones

1. Room permite separar la representación de datos, las operaciones de acceso y la configuración de la base de datos mediante Entity, DAO y Database. También genera el código del DAO y valida las consultas durante la compilación.

2. El uso de un `Flow` desde Room permite que Compose observe los cambios de la tabla y actualice la lista al insertar, actualizar o eliminar usuarios.

3. El alcance de corrutinas del ViewModel vincula las operaciones a su ciclo de vida, y el dispatcher de entrada/salida mantiene el trabajo de base de datos fuera del hilo principal.

4. La migración destructiva permite aplicar rápidamente el cambio al esquema de versión 2; en una app que deba conservar los datos existentes se debe definir una migración explícita.

Capítulo 4

Cuestionario

1. Función de `@Dao`

`@Dao` identifica una interfaz o clase que declara las operaciones para insertar, actualizar, eliminar o consultar datos. Room genera su implementación en compilación y valida las consultas SQL. Frente a usar directamente `SQLiteOpenHelper`, reduce código repetitivo y permite encontrar errores de consulta durante la compilación.

2. Clave primaria autogenerada con valor 0

Con `@PrimaryKey(autoGenerate = true)` y una clave de tipo `Int`, Room trata el valor `0` como "sin asignar". SQLite genera un identificador único al insertar la fila y el registro queda guardado con ese identificador.

3. DAO, ViewModel y Dispatchers.IO

`viewModelScope` vincula el trabajo al ciclo de vida del ViewModel. `Dispatchers.IO` está diseñado para tareas de entrada/salida, como leer y escribir en almacenamiento. Si se ejecutara una operación bloqueante en el hilo principal, la interfaz podría congelarse y Android podría mostrar un ANR. Las funciones suspend de Room son seguras para orquestarse desde corrutinas; el laboratorio usa `Dispatchers.IO` explícitamente para mantener clara la separación del trabajo de base de datos.

4. Propósito de heredar de `Application`

`Application` permite mantener estado compartido durante la vida del proceso de la aplicación y su `onCreate()` se ejecuta antes de crear las actividades. `RoomApplication` inicializa una instancia de `AppDatabase` con el contexto de aplicación para que los componentes puedan acceder al mismo punto de entrada.

5. Función de KSP con Room

KSP (Kotlin Symbol Processing) procesa anotaciones y símbolos Kotlin durante la compilación. Room lo utiliza para analizar `UserEntity`, `UserDao` y `AppDatabase`, generar la implementación de acceso a datos y detectar inconsistencias de consultas antes de ejecutar la aplicación.

Capítulo 5

Comprobación de compilación

Se compiló la variante de depuración con:

```
.\gradlew.bat :app:assembleDebug
```

Resultado: `BUILD SUCCESSFUL`. El APK de depuración se genera en: `app/build/outputs/apk/debug/app-debug.apk`. La aplicación se instaló y se abrió en el AVD Pixel 7 Pro; se comprobó que el Worker carga el lote de usuarios y que la búsqueda por apellido filtra la lista.

Capítulo 6

Referencias bibliográficas

1. Android Developers (2026). *Ciclo de vida de la actividad*. https://developer.android.com/guide/components/activities/activity-lifecycle
2. Android Developers (2026). *Guía de WorkManager: programación de tareas asíncronas*. https://developer.android.com/develop/background-work/background-tasks/persistent/getting-started
3. Kotlin Foundation (2025). *Coroutines Guide: Dispatchers and Context*. https://kotlinlang.org/docs/coroutines-basics.html
4. Android Developers (2026). *Common Architecture Patterns: MVVM*. https://developer.android.com/topic/architecture
5. Android Developers (2026). *Room: guardar datos en una base de datos local*. https://developer.android.com/training/data-storage/room
6. Android Developers (2026). *Migrar de kapt a KSP*. https://developer.android.com/build/migrate-to-ksp